

Tecnologias e Serviços Multimédia

TRABALHO PRÁTICO Nº 1

Mestrado Integrado em Engenharia de Telecomunicações e Informática
José Eduardo da Silva Santos – a82350



UNIVERSIDADE DO MINHO | ANO LETIVO 2019/2020



Índice

Índice	2
Índice de figuras	2
Índice de tabelas.....	2
1 Introdução	3
2 Compressão	3
3 Descompressão.....	5
4 Testes	7
5 Conclusão	12

Índice de figuras

Figura 2.1 – Estrutura do cabeçalho para menos de 129 símbolos diferentes.....	4
Figura 2.2 – Estrutura do cabeçalho para mais de 128 símbolos diferentes.	4
Figura 3.1 – Exemplo do símbolo ‘A’ de código 1010 na árvore binária.....	6
Figura 4.1 – Tabela de Shannon-Fano da compressão do último bloco do ficheiro TXT.....	7
Figura 4.2 – Tabela de Shannon-Fano da compressão do último bloco do ficheiro TXT (cont.).....	8
Figura 4.3 – Tabela de Shannon-Fano da descompressão do último bloco do ficheiro TXT.....	9
Figura 4.4 – Compressão com ocultação das tabelas de Shannon-Fano.	10
Figura 4.5 – Compressão com ocultação das mensagens dos passos intermédios.	10
Figura 4.6 – Compressão, descompressão e validação de todos os ficheiros de teste.	11

Índice de tabelas

Tabela 4.1 – Resultados da compressão e descompressão.	11
---	----

1 Introdução

Neste trabalho prático, foi desenvolvido um *software* para compressão e outro de descompressão de quaisquer tipos e tamanhos de ficheiros usando o algoritmo de Shannon-Fano, assim como outras técnicas de otimização da eficiência e rapidez de compressão/descompressão.

Neste relatório encontra-se a descrição dos métodos de implementação dos algoritmos e das técnicas usadas.

2 Compressão

Para possibilitar a compressão de ficheiros com dezenas de Megabytes, foi necessária a implementação de blocos independentes, pois o sistema operativo não aceita o armazenamento de muitas dezenas em memória do programa para gerar uma só tabela Shannon-Fano. Para tal, optou-se por 1 MB (1 048 576 bytes) para o tamanho de cada bloco lido pelo ficheiro a comprimir.

Ao ser lido um bloco no ficheiro, é seguido o seguinte pseudo-algoritmo:

1. Uma estrutura que armazena dados estatísticos de um símbolo tem a sua frequência incrementada por cada vez que o símbolo aparece no bloco.
2. A frequência do símbolo mais frequente é recolhida.
3. Enquanto a frequência máxima for superior a 255, todas as frequências são divididas a meio.

Isto permite que cada frequência caiba num só byte, sendo as proporções entre símbolos mantidas.

4. As estruturas dos símbolos são ordenadas por ordem decrescente de frequência.
5. A partir da probabilidade de cada símbolo, é usada a seguinte fórmula para calcular a entropia do bloco:

$$H(X) = \sum_{i=0}^{N-1} \left(\frac{\text{frequência}_i}{\text{total}} \times \log_2 \left(\frac{\text{total}}{\text{frequência}_i} \right) \right)$$

6. A tabela de Shannon-Fano é construída recursivamente:
 - 1.1. A primeira chamada à função recebe todos os símbolos, o número de símbolos diferentes e o tamanho do bloco.
 - 1.2. Os elementos recebidos são divididos em dois grupos, de modo a que tenham o tamanho mais próximo possível, tendo a probabilidade como peso.
 - 1.3. Os elementos do primeiro grupo terão o *bit* 0 concatenado ao respetivo código e os do segundo terão o *bit* 1.
 - 1.4. Enquanto cada grupo tiver mais do que um elemento, esta função é novamente chamada, recursivamente, para cada grupo, enviando os respetivos elementos, número de símbolos diferentes e tamanho do grupo.

7. A tabela obtida é reordenada de modo a que os comprimentos dos códigos Shannon-Fano sejam crescentes enquanto as respectivas frequências são decrescentes.

Há raros casos em que o código de um símbolo é maior do que o de um símbolo menos frequente. Para que haja melhor compressão, este caso não deve acontecer pois pretende-se que os símbolos mais frequentes ocupem menos espaço no ficheiro comprimido.

8. A tabela calculada é apresentada ao utilizador (símbolo, probabilidade e código), assim como a entropia e o comprimento médio, calculado com a seguinte fórmula:

$$\bar{N} = \sum_{i=0}^{N-1} \left(\frac{\text{frequência}_i}{\text{total}} \times \text{comprimento_código}_i \right)$$

9. Para preencher o ficheiro comprimido, é em primeiro lugar criado um cabeçalho com estrutura dependente do número de símbolos diferentes:

Se o número de símbolos for inferior a 129:

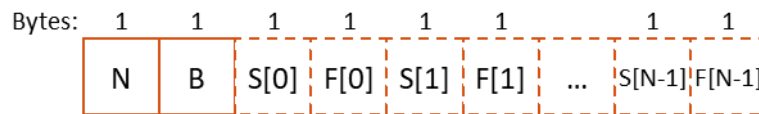


Figura 2.1 – Estrutura do cabeçalho para menos de 129 símbolos diferentes.

- **N** – número de símbolos diferentes;
- **B** – número de *bits* usados no último *byte* do bloco;
- **S[n]** – valor do símbolo;
- **F[n]** – frequência de S[n].

Como cada símbolo irá ocupar 2 bytes e o número máximo de símbolos é 128, o tamanho do cabeçalho será $2 + N \times 2$, variando entre 2 e 258.

Se o número de símbolos for superior a 127:

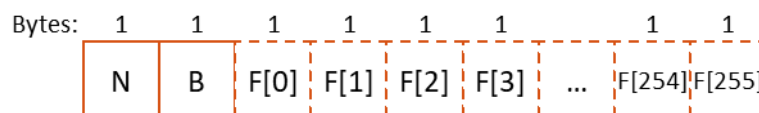


Figura 2.2 – Estrutura do cabeçalho para mais de 128 símbolos diferentes.

- **F[n]** – frequência do símbolo de valor *n*.
Nesta estrutura, como cada símbolo (existente ou não em *N*) ocupa apenas 1 byte (frequência, sem identificação) o tamanho do cabeçalho é sempre 258.

10. Voltando a ler o bloco original para saber as posições de cada símbolo, o ficheiro comprimido será escrito com a mesma ordem dessas posições, mas com símbolos binários diferentes dos 8 bits que representam o valor de cada símbolo. Os símbolos usados são os obtidos na tabela Shannon-Fano.
11. Depois de comprimido um bloco, o processo é repetido no seguinte, até o ficheiro original ser todo lido.

3 Descompressão

A descompressão é feita também de modo a cada bloco ser descomprimido independentemente; contudo, os blocos comprimidos têm tamanhos variados, portanto não se consegue separá-los à primeira vista. Tendo de ser lida uma quantidade de *bytes* suficiente X para conter pelo menos um bloco comprimido; naturalmente X conterá *bytes* de blocos seguintes, sendo desconhecido o início dos mesmos. Para se descobrir, os *bytes* recolhidos são comprimidos até serem obtidos N bytes descomprimidos, sendo N o tamanho definido para cada bloco. Os *bytes* que sobraem serão aproveitados na restante leitura do ficheiro.

O pseudo-algoritmo da descompressão para cada X bytes lidos é o seguinte:

1. O cabeçalho do bloco é lido de modo a determinar os símbolos existentes e as frequências dos mesmos, que irão preencher estruturas semelhantes às referidas na compressão.
2. Sabendo as frequências de cada símbolo, estes são ordenados, a entropia é calculada, assim como a tabela de Shannon-Fano, e a mesma é ordenada de acordo com os comprimentos de código, usando os mesmo métodos da compressão.
3. A tabela calculada é apresentada ao utilizador, assim como a entropia e o comprimento médio.
4. Pretende-se agora traduzir os *bits* do ficheiro comprimido nos símbolos obtidos na tabela de Shannon-Fano. Como procurar *bit* a *bit* uma correspondência demoraria uma enorme quantidade de tempo para certos tamanhos de ficheiro, foi usada a procura através de árvores binárias.
 - 4.1. A árvore binária contém para cada nó um símbolo correspondente e apontadores para o nó zero e nó um; cada símbolo corresponderá a uma corrente de nós de acordo com o seu código de Shannon-Fano.

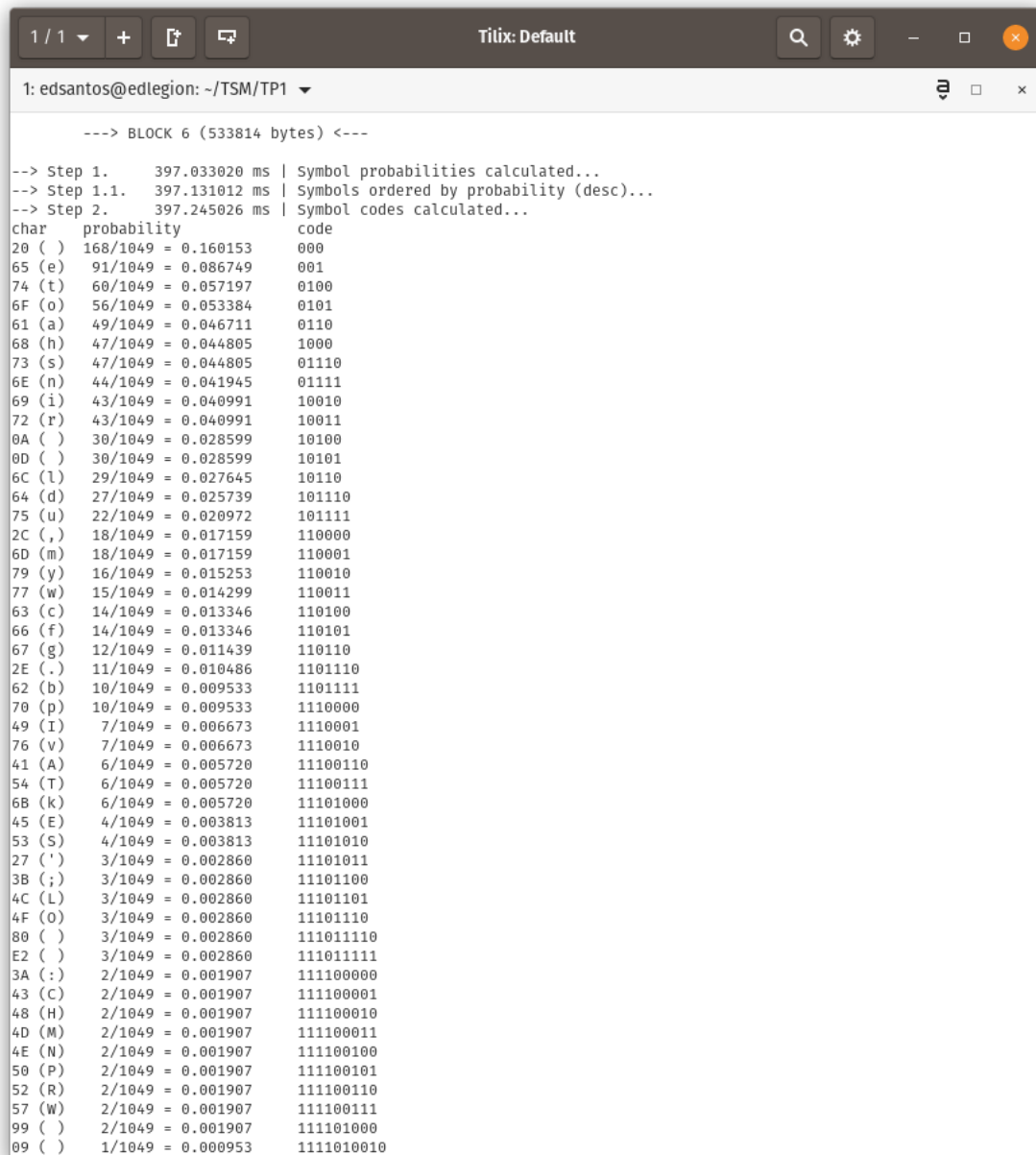
Por exemplo:

Símbolo: A, código: 1010 – corrente: *root* -> *one* -> *zero* -> *one* -> *zero* -> *symbol* = 'A'. Esquematizado na Figura 3.1.

4 Testes

Para testar o funcionamento dos programas, foram usados os ficheiros de teste disponibilizados pelo professor que contém tanto ficheiros de texto como de áudio, dos 5 MB aos 92 MB, sendo um dos ficheiros impossível de ser comprimido.

Na Figura 4.1 e Figura 4.2 encontra-se o resultado de compressão do ficheiro texto.txt com a tabela de Shannon-Fano do último bloco.



```
1: edsantos@edlegion: ~/TSM/TP1
---> BLOCK 6 (533814 bytes) <---

--> Step 1.      397.033020 ms | Symbol probabilities calculated...
--> Step 1.1.    397.131012 ms | Symbols ordered by probability (desc)...
--> Step 2.      397.245026 ms | Symbol codes calculated...

char  probability      code
20 ( ) 168/1049 = 0.160153 000
65 (e)  91/1049 = 0.086749 001
74 (t)  60/1049 = 0.057197 0100
6F (O)  56/1049 = 0.053384 0101
61 (a)  49/1049 = 0.046711 0110
68 (h)  47/1049 = 0.044805 1000
73 (s)  47/1049 = 0.044805 01110
6E (n)  44/1049 = 0.041945 01111
69 (i)  43/1049 = 0.040991 10010
72 (r)  43/1049 = 0.040991 10011
0A ( )  30/1049 = 0.028599 10100
0D ( )  30/1049 = 0.028599 10101
6C (l)  29/1049 = 0.027645 10110
64 (d)  27/1049 = 0.025739 101110
75 (u)  22/1049 = 0.020972 101111
2C (,)  18/1049 = 0.017159 110000
6D (m)  18/1049 = 0.017159 110001
79 (y)  16/1049 = 0.015253 110010
77 (w)  15/1049 = 0.014299 110011
63 (c)  14/1049 = 0.013346 110100
66 (f)  14/1049 = 0.013346 110101
67 (g)  12/1049 = 0.011439 110110
2E (.)  11/1049 = 0.010486 1101110
62 (b)  10/1049 = 0.009533 1101111
70 (p)  10/1049 = 0.009533 1110000
49 (I)   7/1049 = 0.006673 1110001
76 (v)   7/1049 = 0.006673 1110010
41 (A)   6/1049 = 0.005720 11100110
54 (T)   6/1049 = 0.005720 11100111
68 (k)   6/1049 = 0.005720 11101000
45 (E)   4/1049 = 0.003813 11101001
53 (S)   4/1049 = 0.003813 11101010
27 (')   3/1049 = 0.002860 11101011
3B (;)   3/1049 = 0.002860 11101100
4C (L)   3/1049 = 0.002860 11101101
4F (O)   3/1049 = 0.002860 11101110
80 ( )   3/1049 = 0.002860 111011110
E2 ( )   3/1049 = 0.002860 111011111
3A (: )  2/1049 = 0.001907 111100000
43 (C)   2/1049 = 0.001907 111100001
48 (H)   2/1049 = 0.001907 111100010
4D (M)   2/1049 = 0.001907 111100011
4E (N)   2/1049 = 0.001907 111100100
50 (P)   2/1049 = 0.001907 111100101
52 (R)   2/1049 = 0.001907 111100110
57 (W)   2/1049 = 0.001907 111100111
99 ( )   2/1049 = 0.001907 111101000
09 ( )   1/1049 = 0.000953 1111010010
```

Figura 4.1 – Tabela de Shannon-Fano da compressão do último bloco do ficheiro TXT.


```

1: edsantos@edlegion: ~/TSM/TP1
21 (!) 1/1049 = 0.000953 1111010011
22 (") 1/1049 = 0.000953 1111010100
24 ($) 1/1049 = 0.000953 1111010101
25 (%) 1/1049 = 0.000953 1111010110
26 (6) 1/1049 = 0.000953 1111010111
28 (() 1/1049 = 0.000953 1111011000
29 ()) 1/1049 = 0.000953 1111011001
2A (*) 1/1049 = 0.000953 1111011010
2D (-) 1/1049 = 0.000953 1111011011
2F (/) 1/1049 = 0.000953 1111011100
30 (0) 1/1049 = 0.000953 1111011101
31 (1) 1/1049 = 0.000953 1111011110
32 (2) 1/1049 = 0.000953 1111100000
33 (3) 1/1049 = 0.000953 1111100001
34 (4) 1/1049 = 0.000953 1111100010
35 (5) 1/1049 = 0.000953 1111100011
36 (6) 1/1049 = 0.000953 1111100100
37 (7) 1/1049 = 0.000953 1111100101
38 (8) 1/1049 = 0.000953 1111100110
39 (9) 1/1049 = 0.000953 1111100111
3F (?) 1/1049 = 0.000953 1111101000
40 (a) 1/1049 = 0.000953 1111101001
42 (B) 1/1049 = 0.000953 1111101010
44 (D) 1/1049 = 0.000953 1111101011
46 (F) 1/1049 = 0.000953 1111101100
47 (G) 1/1049 = 0.000953 1111101101
4A (J) 1/1049 = 0.000953 1111101110
4B (K) 1/1049 = 0.000953 1111110000
51 (Q) 1/1049 = 0.000953 1111110001
55 (U) 1/1049 = 0.000953 1111110010
56 (V) 1/1049 = 0.000953 1111110011
58 (X) 1/1049 = 0.000953 1111110100
59 (Y) 1/1049 = 0.000953 1111110101
5A (Z) 1/1049 = 0.000953 1111110110
5B (L) 1/1049 = 0.000953 1111110111
5C (\) 1/1049 = 0.000953 1111111000
5D (I) 1/1049 = 0.000953 1111111001
5F (.) 1/1049 = 0.000953 1111111010
6A (j) 1/1049 = 0.000953 1111111011
71 (q) 1/1049 = 0.000953 1111111100
78 (x) 1/1049 = 0.000953 1111111101
7A (z) 1/1049 = 0.000953 1111111110
94 ( ) 1/1049 = 0.000953 11110111110
98 ( ) 1/1049 = 0.000953 11110111111
9C ( ) 1/1049 = 0.000953 11111011110
9D ( ) 1/1049 = 0.000953 11111011111
A6 ( ) 1/1049 = 0.000953 11111111110
C3 ( ) 1/1049 = 0.000953 11111111111
H(X) = 4.930068 | N = 4.978074
--> Step 3. 397.682007 ms | Symbols assimilated...
--> Step 4. 434.287018 ms | Block compressed | 533814 -> 325614 bytes (39.002 %)
--> Complete. 434.301025 ms | File 'texto.txt' compressed -> 'texto.txt.sf' | 5776694 -> 3528271 bytes (38.922 %)
edsantos@edlegion:~/TSM/TP1$

```

Figura 4.2 – Tabela de Shannon-Fano da compressão do último bloco do ficheiro TXT (cont.).

A mesma tabela encontra-se no lado do descompressor na Figura 4.3.

```
1 / 1 + TiliX: Default
1: edsantos@edlegion: ~/TSM/TP1
21 (!) 1/1049 = 0.000953 1111010011
22 (") 1/1049 = 0.000953 1111010100
24 ($) 1/1049 = 0.000953 1111010101
25 (%) 1/1049 = 0.000953 1111010110
26 (6) 1/1049 = 0.000953 1111010111
28 (( ) 1/1049 = 0.000953 1111011000
29 ()) 1/1049 = 0.000953 1111011001
2A (*) 1/1049 = 0.000953 1111011010
2D (-) 1/1049 = 0.000953 1111011011
2F (/) 1/1049 = 0.000953 1111011100
30 (0) 1/1049 = 0.000953 1111011101
31 (1) 1/1049 = 0.000953 1111011110
32 (2) 1/1049 = 0.000953 1111100000
33 (3) 1/1049 = 0.000953 1111100001
34 (4) 1/1049 = 0.000953 1111100010
35 (5) 1/1049 = 0.000953 1111100011
36 (6) 1/1049 = 0.000953 1111100100
37 (7) 1/1049 = 0.000953 1111100101
38 (8) 1/1049 = 0.000953 1111100110
39 (9) 1/1049 = 0.000953 1111100111
3F (?) 1/1049 = 0.000953 1111101000
40 (a) 1/1049 = 0.000953 1111101001
42 (B) 1/1049 = 0.000953 1111101010
44 (D) 1/1049 = 0.000953 1111101011
46 (F) 1/1049 = 0.000953 1111101100
47 (G) 1/1049 = 0.000953 1111101101
4A (J) 1/1049 = 0.000953 1111101110
4B (K) 1/1049 = 0.000953 1111110000
51 (Q) 1/1049 = 0.000953 1111110001
55 (U) 1/1049 = 0.000953 1111110010
56 (V) 1/1049 = 0.000953 1111110011
58 (X) 1/1049 = 0.000953 1111110100
59 (Y) 1/1049 = 0.000953 1111110101
5A (Z) 1/1049 = 0.000953 1111110110
5B (L) 1/1049 = 0.000953 1111110111
5C (\) 1/1049 = 0.000953 1111111000
5D (I) 1/1049 = 0.000953 1111111001
5F ( _ ) 1/1049 = 0.000953 1111111010
6A (j) 1/1049 = 0.000953 1111111011
71 (q) 1/1049 = 0.000953 1111111100
78 (x) 1/1049 = 0.000953 1111111101
7A (z) 1/1049 = 0.000953 1111111110
94 ( ) 1/1049 = 0.000953 11110111110
98 ( ) 1/1049 = 0.000953 11110111111
9C ( ) 1/1049 = 0.000953 11111011110
9D ( ) 1/1049 = 0.000953 11111011111
A6 ( ) 1/1049 = 0.000953 11111111110
C3 ( ) 1/1049 = 0.000953 11111111111
H(X) = 4.930068 | N = 4.978074
--> Step 3. 294.154999 ms | Symbols assimilated...
--> Step 4. 324.127014 ms | Block decompressed -> 533814 bytes

--> Complete. 324.159027 ms | File 'texto.txt.sf' decompressed -> 'decomp.txt' (5776694 bytes)
edsantos@edlegion:~/TSM/TP1$
```

Figura 4.3 – Tabela de Shannon-Fano da descompressão do último bloco do ficheiro TXT.

Como meio de otimização na rapidez de compressão/descompressão, o utilizador pode optar por ocultar as tabelas com o argumento ‘h1’ (Figura 4.4), ou até mesmo todas as mensagens exceto as do início e fim de compressão/descompressão com o argumento ‘h2’ (Figura 4.5).

```
1 / 1 + TiliX: Default
1: edsantos@edlegion: ~/TSM/TP1
edsantos@edlegion:~/TSM/TP1$ ./compress texto.txt texto.txt.sf h1
--> Compressing 'texto.txt'...

---> BLOCK 1 (1048576 bytes) <---
--> Step 1. 7.799000 ms | Symbol probabilities calculated...
--> Step 1.1. 7.876000 ms | Symbols ordered by probability (desc)...
--> Step 2. 7.994000 ms | Symbol codes calculated...
--> Step 3. 7.998000 ms | Symbols assimilated...
--> Step 4. 79.653008 ms | Block compressed | 1048576 -> 640218 bytes (38.944 %)

---> BLOCK 2 (1048576 bytes) <---
--> Step 1. 87.159004 ms | Symbol probabilities calculated...
--> Step 1.1. 87.215004 ms | Symbols ordered by probability (desc)...
--> Step 2. 87.288002 ms | Symbol codes calculated...
--> Step 3. 87.292007 ms | Symbols assimilated...
--> Step 4. 156.588013 ms | Block compressed | 1048576 -> 639741 bytes (38.990 %)

---> BLOCK 3 (1048576 bytes) <---
--> Step 1. 163.792007 ms | Symbol probabilities calculated...
--> Step 1.1. 163.834015 ms | Symbols ordered by probability (desc)...
--> Step 2. 163.944000 ms | Symbol codes calculated...
--> Step 3. 163.948013 ms | Symbols assimilated...
--> Step 4. 235.227005 ms | Block compressed | 1048576 -> 639948 bytes (38.970 %)

---> BLOCK 4 (1048576 bytes) <---
--> Step 1. 242.304016 ms | Symbol probabilities calculated...
--> Step 1.1. 242.346008 ms | Symbols ordered by probability (desc)...
--> Step 2. 242.422012 ms | Symbol codes calculated...
--> Step 3. 242.426010 ms | Symbols assimilated...
--> Step 4. 311.696014 ms | Block compressed | 1048576 -> 637634 bytes (39.190 %)

---> BLOCK 5 (1048576 bytes) <---
--> Step 1. 318.710022 ms | Symbol probabilities calculated...
--> Step 1.1. 318.751007 ms | Symbols ordered by probability (desc)...
--> Step 2. 318.808014 ms | Symbol codes calculated...
--> Step 3. 318.812012 ms | Symbols assimilated...
--> Step 4. 388.534027 ms | Block compressed | 1048576 -> 645116 bytes (38.477 %)

---> BLOCK 6 (533814 bytes) <---
--> Step 1. 392.515015 ms | Symbol probabilities calculated...
--> Step 1.1. 392.567017 ms | Symbols ordered by probability (desc)...
--> Step 2. 392.645020 ms | Symbol codes calculated...
--> Step 3. 392.649017 ms | Symbols assimilated...
--> Step 4. 428.828033 ms | Block compressed | 533814 -> 325614 bytes (39.002 %)

--> Complete. 428.881012 ms | File 'texto.txt' compressed -> 'texto.txt.sf' | 5776694 -> 3528271 bytes (38.922 %)
edsantos@edlegion:~/TSM/TP1$
```

Figura 4.4 – Compressão com ocultação das tabelas de Shannon-Fano.

```
1 / 1 + TiliX: Default
1: edsantos@edlegion: ~/TSM/TP1
edsantos@edlegion:~/TSM/TP1$ ./compress texto.txt texto.txt.sf h2
--> Compressing 'texto.txt'...
--> Complete. 421.574005 ms | File 'texto.txt' compressed -> 'texto.txt.sf' | 5776694 -> 3528271 bytes (38.922 %)
edsantos@edlegion:~/TSM/TP1$
```

Figura 4.5 – Compressão com ocultação das mensagens dos passos intermédios.

Na Figura 4.6 encontra-se o *snapshot* da compressão e descompressão de todos os ficheiros de teste (texto.txt, music.mp3, music.flac e music.wav), assim como a confirmação de que os ficheiros descomprimidos são iguais aos originais (através do comando diff).



```
1: edsantos@edlegion: ~/TSM/TP1
edsantos@edlegion:~/TSM/TP1$ ./compress texto.txt texto.txt.sf h2
--> Compressing 'texto.txt'...
--> Complete. 421.574005 ms | File 'texto.txt' compressed -> 'texto.txt.sf' | 5776694 -> 3528271 bytes (38.922 %)
edsantos@edlegion:~/TSM/TP1$ ./compress music.mp3 music.mp3.sf h2
--> Compressing 'music.mp3'...
--> Complete. 2658.397217 ms | File 'music.mp3' compressed -> 'music.mp3.sf' | 21058294 -> 21015998 bytes (0.201 %)
edsantos@edlegion:~/TSM/TP1$ ./compress music.flac music.flac.sf h2
--> Compressing 'music.flac'...
--> Complete. 6793.178223 ms | File 'music.flac' compressed -> 'music.flac.sf' | 53380401 -> 53507892 bytes (-0.239 %)
edsantos@edlegion:~/TSM/TP1$ ./compress music.wav music.wav.sf h2
--> Compressing 'music.wav'...
--> Complete. 11317.016602 ms | File 'music.wav' compressed -> 'music.wav.sf' | 92847804 -> 90291931 bytes (2.753 %)
edsantos@edlegion:~/TSM/TP1$

2: edsantos@edlegion: ~/TSM/TP1
edsantos@edlegion:~/TSM/TP1$ ./decompress texto.txt.sf decomp.txt h2
--> Decompressing 'texto.txt.sf'...
--> Complete. 296.685028 ms | File 'texto.txt.sf' decompressed -> 'decomp.txt' (5776694 bytes)
edsantos@edlegion:~/TSM/TP1$ ./decompress music.mp3.sf decomp.mp3 h2
--> Decompressing 'music.mp3.sf'...
--> Complete. 1958.207153 ms | File 'music.mp3.sf' decompressed -> 'decomp.mp3' (21058294 bytes)
edsantos@edlegion:~/TSM/TP1$ ./decompress music.flac.sf decomp.flac h2
--> Decompressing 'music.flac.sf'...
--> Complete. 4993.043457 ms | File 'music.flac.sf' decompressed -> 'decomp.flac' (53380401 bytes)
edsantos@edlegion:~/TSM/TP1$ ./decompress music.wav.sf decomp.wav h2
--> Decompressing 'music.wav.sf'...
--> Complete. 8134.930176 ms | File 'music.wav.sf' decompressed -> 'decomp.wav' (92847804 bytes)
edsantos@edlegion:~/TSM/TP1$ diff decomp.txt texto.txt
edsantos@edlegion:~/TSM/TP1$ diff decomp.mp3 music.mp3
edsantos@edlegion:~/TSM/TP1$ diff decomp.flac music.flac
edsantos@edlegion:~/TSM/TP1$ diff decomp.wav music.wav
edsantos@edlegion:~/TSM/TP1$
```

Figura 4.6 – Compressão, descompressão e validação de todos os ficheiros de teste.

Estes resultados estão agrupados na Tabela 4.1.

Tabela 4.1 – Resultados da compressão e descompressão.

Ficheiro	Original (bytes)	Comprimido (bytes)	Compressão (ms)	Descompressão (ms)	Compressão (%)
texto.txt	5 777 694	3 528 271	422	297	38,922
music.mp3	21 058 294	21 015 998	2 658	1 958	0,201
music.flac	53 380 401	53 507 892	6 793	4 993	-0,239
music.wav	92 284 804	90 291 931	11 317	8 135	2,753

Como se pode verificar, não foi possível comprimir o ficheiro music.flac. Isto deve-se à natureza da própria extensão pois FLAC é um tipo de ficheiro de áudio já comprimido, portanto tem uma entropia elevada. O MP3 também é um tipo comprimido, mas não o suficiente para não poder ser mais comprimido, apesar de ser apenas 0,2 %. A compressão deste ficheiro só foi conseguida graças à redistribuição dos códigos de Shannon-Fano para estes terem comprimento crescente com as frequências decrescentes.

5 Conclusão

Neste trabalho prático foram colocados à prática conhecimentos adquiridos nas aulas teóricas (e aos acetatos das mesmas) e até mesmo na UC (Unidade Curricular) de Codificação e Transmissão do semestre passado, sendo em ambos referido o algoritmo de Shannon-Fano. Contudo só neste trabalho é que se testou a eficiência deste algoritmo, que em teoria devia comprimir qualquer tipo de ficheiro pois o comprimento médio nunca deve ultrapassar o valor de 8, correspondente ao comprimento binário base de um símbolo. Infelizmente havia algumas situações em que esta condição era ligeiramente ultrapassada, impedindo a compressão do ficheiro FLAC.

Um algoritmo de compressão adicional planeado foi o RLE (*Run-length Encoding*), contudo ao ser testado ao *bit* com contadores de 4 bits e até 3 iterações, não se verificou resultados de compressão. Tanto este problema como a falta de tempo impediram a continuação desta implementação, tendo de ser descartado.

Em suma, para além das inconveniências que surgiram, penso que concluí este trabalho prático com sucesso.